# $B^+$-Tree Insertion: Explanation and Examples

## 1 Introduction

Insertion in a $B^+$-tree ensures the tree remains balanced and supports efficient search and range queries. A single insertion may lead to:

- Leaf node split
- Internal node split
- Height increase (new root)

## 2 Initial Tree

### Before Insertion

Consider the following $B^+$-tree before inserting the key "Adams":

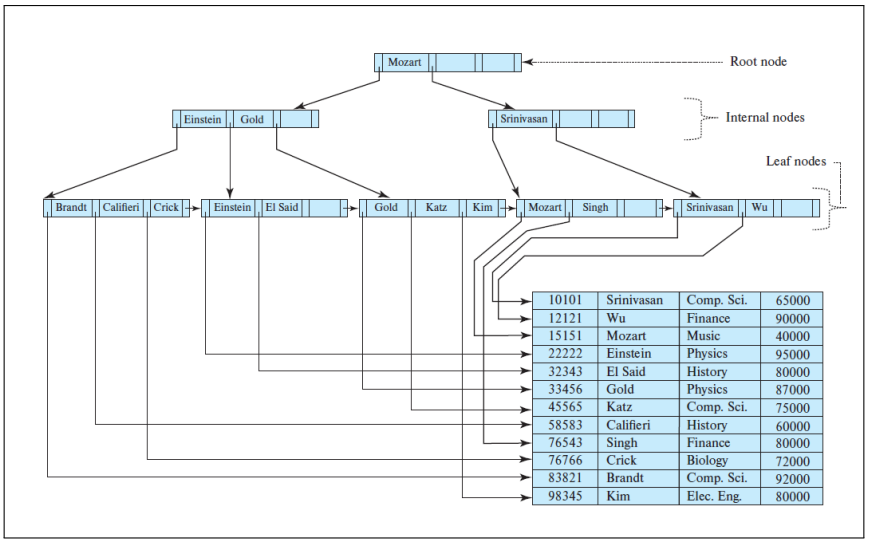


Figure 1: Initial $B^+$-tree before insertion of "Adams"

# 3 Example 1: Inserting “Adams”

**Step-by-Step Insertion of “Adams”**

1. Locate the leaf node containing “Brandt,” “Califieri,” “Crick”.
2. **Determine position alphabetically:** “Adams” is compared with existing keys and placed in sorted order. Since “Adams” $<$ “Brandt,” it goes before “Brandt” in the leaf.
3. Leaf is full $\Rightarrow$ split required.
4. Redistribute keys:
   - Left leaf: “Adams,” “Brandt”
   - Right leaf: “Califieri,” “Crick”
5. Promote the smallest key of the new node (“Califieri”) to the parent.
6. If parent overflows, split recursively up to the root.

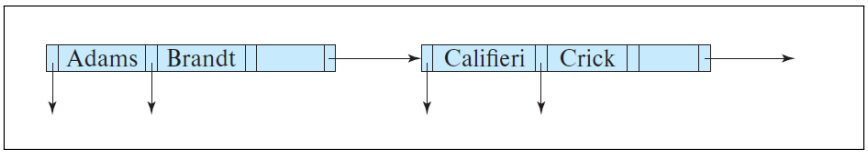


Figure 2: Leaf node split on insertion of “Adams”

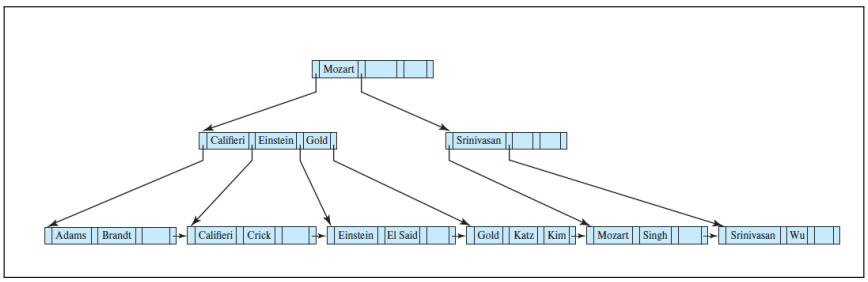


Figure 3: Resulting $B^+$-tree after inserting “Adams”

# 4 Example 2: Inserting "Lamport"

**Non-Leaf Node Split**

1. Determine the correct leaf node for "Lamport" based on alphabetical order: Compare "Lamport" with existing keys in the leaf and insert it in sorted order.
2. Leaf splits (e.g., "Kim" and "Lamport" go to new right leaf) because the leaf is full.
3. Smallest key of the new leaf ("Kim") is inserted into the parent to act as a separator.
4. Parent may split if full:
   - Keys between left and right children (e.g., "Gold") are promoted to parent.
5. Splits may propagate upward, maintaining $B^+$-tree balance.

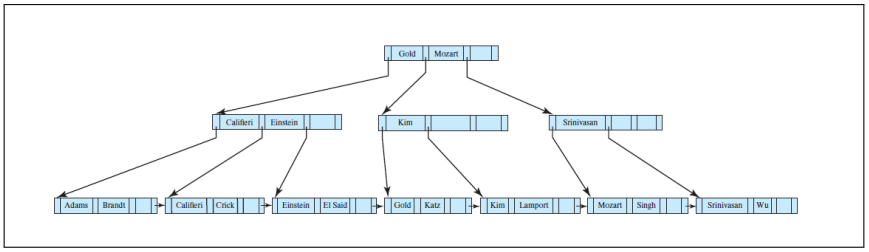


Figure 4: Insertion of "Lamport" causing parent split

# 5 General Insertion Algorithm

**Algorithm: Insertion into $B^+$-Tree**

**procedure** `insert`(key $K$, pointer $P$):

1. If tree is empty, create a new leaf node $L$ as root.
2. Find leaf $L$ where $K$ belongs.
3. If $L$ has space, insert $(K, P)$.
4. If $L$ is full:
   - 4.1. Split $L$ into two nodes.
   - 4.2. Redistribute keys.
   - 4.3. Insert new node into parent.
   - 4.4. Recursively split parent if necessary.

**Key Insights**

- Leaf splits distribute actual data entries across two nodes.
- Non-leaf splits redistribute pointers and promote a key to the parent.
- Recursive splitting ensures the tree remains balanced and maintains all $B^+$-tree properties.

# 6 Conclusion

- $B^+$-tree insertion guarantees logarithmic search cost.
- May involve multiple splits, including internal nodes, and occasionally a new root.
- Ensures balanced tree structure, supporting efficient point and range queries.
- Sequential leaf chaining facilitates range queries and ordered traversal.